

Comparison of CBOW and Skip-gram Architectures

Kira Maslennikova DS-01
Innopolis university

Motivation

Word embeddings are a fundamental component of many NLP systems, yet different training architectures may produce embeddings with varying properties. This study compares two Word2Vec architectures: Continuous Bag-of-Words (CBOW) and Skip-gram — focusing on the quality of learned representations. I investigate how these models differ in capturing semantic and syntactic relationships, handling frequent versus rare words, and performing in downstream tasks. Understanding these differences helps guide the choice of embedding models under different data and computational constraints.

Experimental Setup

Dataset:
Simple English
Wikipedia corpus

Evaluation benchmarks:
- SimLex-999
- Google Analogies dataset
- Downstream classification task

Models:
3 CBOW & 3 Skip-gram configurations

Additional: Vocabulary split into frequent and rare subsets

Results

Training Efficiency:

Skip-gram is about 3–4 times slower than CBOW. CBOW's learning speed does not change much with hyperparameter tuning, which cannot be said about Skip-Gram. (Figure 1: Training time Comparison)

Evaluation results (Table1):

- Skip-gram models consistently outperform CBOW on semantic tasks (SimLex and analogies).
- CBOW performs comparably on syntactic tasks but struggles with semantic relationships.
- Skip-gram demonstrates stronger performance on rare words, as reflected in higher embedding density.
- Classification results show that Skip-gram provides better general-purpose representations.

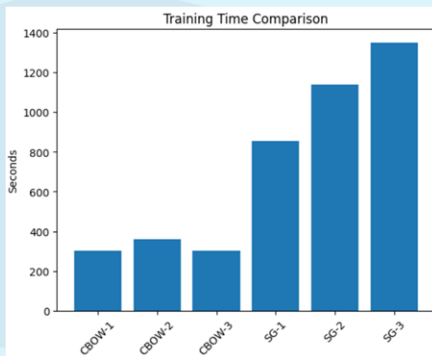


Figure 1

Nearest neighbor

inspection reveals that:
- Skip-gram captures more semantically meaningful relationships
- CBOW tends to focus on more frequent and syntactically related words

Visualization (PCA):

- Skip-gram embeddings form more distinct semantic clusters
- CBOW embeddings appear more compact but less separable

Hyperparameter effects:

- Larger vector size (200) consistently improves performance across similarity, analogies, and classification tasks (CBOW-2, SG-2).
- Larger context window (10) improves semantic understanding (best results in semantic analogies: CBOW-3, SG-3).
- Smaller window (5) is better for syntactic tasks, where local context is important (CBOW-2, SG-2).

	SimLex_Corr	Semantic_Acc	Syntactic_Acc	Classification_Acc	Freq_Density	Rare_Density	Gap	Time_sec
CBOW-1	0.2346	0.3800	0.4182	0.7526	0.7261	0.7953	-0.0692	301
CBOW-2	0.2492	0.3908	0.4469	0.7681	0.6833	0.7567	-0.0734	359
CBOW-3	0.2374	0.3951	0.4401	0.7668	0.7210	0.7446	-0.0236	301
SG-1	0.2818	0.4355	0.3589	0.7940	0.7502	0.9104	-0.1602	855
SG-2	0.3195	0.4572	0.4024	0.8174	0.6746	0.8883	-0.2137	1136
SG-3	0.2671	0.4626	0.3539	0.8018	0.7479	0.8939	-0.1460	1348

Table1

Methodology

I conducted a controlled comparison of CBOW and Skip-gram architectures using identical preprocessing, dataset, and training conditions.

Models are trained on the common corpus using Word2Vec framework with varying hyperparameters:

- vector size: 100, 200
- context window: 5, 10
- architecture: CBOW vs Skip-gram

Evaluation is performed using both intrinsic and extrinsic methods:

Intrinsic evaluation:

- Word similarity (SimLex-999, Spearman correlation)
- Word analogies (semantic and syntactic accuracy)

Extrinsic evaluation:

- Text classification using averaged embeddings

Additional analysis includes:

- Representation quality for frequent vs rare words
- Nearest neighbor analysis (qualitative evaluation)
- PCA-based embedding visualization

All experiments are conducted under fixed conditions to ensure fair comparison.

Conclusions

Training Efficiency:

- CBOW trains significantly faster than Skip-gram
- Skip-gram requires more computation but learns richer representations

Representation Quality:

- Skip-gram captures semantic relationships more effectively
- CBOW performs well on syntactic patterns but is less expressive

Rare vs Frequent Words:

- Skip-gram provides better representations for rare words
- CBOW is biased toward frequent words

Downstream Performance:

- Skip-gram achieves higher classification accuracy
- CBOW provides competitive but weaker general representations

Hyperparameters

- Larger embeddings (vector size = 200) consistently improve results.
- Larger windows capture semantic context, while smaller windows favor syntactic patterns.
- The best overall configuration is Skip-gram with vector size = 200 and window = 5.

Overall:

- CBOW is preferable when training speed is critical or syntactic patterns are important
- Skip-gram is better suited for tasks requiring high-quality semantic embeddings